

## Übungsblatt 2

### Vorlesung „Secure Software Systems“ Wintersemester 2025/26

#### Praktische Übung Buffer Overflow Exploits

### Stack-Based Buffer Overflow Exploit

In dieser Aufgabe sollen Sie einen Exploit für das Programm `bufovf` schreiben. Das Programm enthält eine Stack-basierte Buffer Overflow Schwachstelle. Nutzen Sie diese aus, um die Return-Adresse zu überschreiben.

#### Aufgabe 1.

- i) Finden Sie die Buffer Overflow Schwachstelle. Wo im Programm ist die Schwachstelle? Welchen Input müssen Sie dem Programm schicken, um den Buffer Overflow auszulösen? Wie lang ist der Input?
- ii) Überschreiben Sie die Return-Adresse zuerst mit dem Wert `0x41414141` (ASCII „AAAA“) um zu demonstrieren, dass Sie den Instruction Pointer kontrollieren können.
- iii) Überschreiben Sie die Return-Adresse mit der Adresse der Funktion `success` (diese finden Sie in der Variablen `retaddr` im Exploit-Template).

### Heap-Based Buffer Overflow Exploit

In dieser Aufgabe sollen Sie einen Exploit für das Programm `heartbeat`<sup>1</sup> schreiben. Dieses Programm enthält eine Heap-basierte Buffer Overflow Schwachstelle. Bei Heap-basierten Buffer Overflows kann man nicht direkt eine Return-Adresse überschreiben, weil diese bekanntlich auf dem Stack liegen. Oftmals befinden sich am Heap jedoch Code-Pointer, die ein Angreifer überschreiben kann (z.B. Pointer auf Funktionen oder Pointer auf Tabellen von virtuellen C++ Methoden). Auch in dem `heartbeat` Programm wird ein Funktionspointer verwendet, der als Ziel für den Buffer Overflow geeignet ist.

#### Aufgabe 2.

- i) Finden Sie die Buffer Overflow Schwachstelle. Wo im Programm ist die Schwachstelle? Welchen Input müssen Sie dem Programm schicken, um den Buffer Overflow auszulösen? Wie lang ist der Input?
- ii) Überschreiben Sie den Funktionspointer zuerst mit dem Wert `0x41414141` (ASCII „AAAA“). Vollziehen Sie den Crash im Debugger nach. Wie werden indirekte Funktionsaufrufe über Funktionspointer auf der Assembler Ebene implementiert? Welche Instruktionen bzw. Register sind involviert?
- iii) Überschreiben Sie den Funktionspointer mit der Adresse der Funktion `success`. Nutzen Sie dazu die Variable `fptraddr` im Exploit-Template. Wie ändert sich der Input für das verwundbare Programm?

---

<sup>1</sup><https://xkcd.com/1354/>

## Code Injection auf x86

Das Programm `sc86` enthält einen Stack-basierten Buffer Overflow. In dieser Aufgabe nutzen Sie diese Schwachstelle aus, um einen Code-Injection Angriff durchzuführen. Ein klassischer Code-Injection Angriff startet eine Betriebssystem-Shell, um so das System weiter zu kompromittieren. Bei Code Injection Angriffen kommt normalerweise ein *Shellcode* zum Einsatz. Das Konzept von Shellcodes kennen wir schon aus vorhergehenden Übungen. Diesmal werden Sie selbst einen Shellcode verfassen und mittels einem Buffer Overflow den Kontrollfluss auf ihren Shellcode umleiten. Beachten Sie, dass bei allen folgenden Übungen jeder Exploit auch ohne Debugger funktionieren sollte. Testen Sie also ihren fertigen Exploit auch ohne `gdb`.

### Aufgabe 3.

- i) Wo befindet sich die Schwachstelle in diesem Programm? Wie können Sie damit die Return-Adresse überschreiben? Wie lang muss der Input sein?
- ii) Erweitern Sie den Shellcode, sodass er eine Betriebssystem-Shell startet. Wie können Sie den Shellcode einschleusen? Ermitteln Sie die Adresse des Shellcode (z.B. mit dem Debugger). An welcher Adresse liegt der eingeschleuste Shellcode? Welche Zeichen darf der Shellcode nicht enthalten?
- iii) Überschreiben Sie die Return-Adresse mit der Adresse des Shellcode. Starten Sie damit eine Shell. Erweitern Sie das Exploit Template `exploit.py` zu einem funktionierenden Exploit. Stellen Sie sicher, dass ihr Exploit auch außerhalb des Debuggers funktionsfähig ist.

## Advanced Shellcoding auf x86 64-Bit

Das 64-Bit x86 Programm `getreal` enthält ebenso einen Stack-basierten Buffer Overflow. Nutzen Sie die Schwachstelle aus, um einen Code-Injection Angriff durchzuführen. Ziel des Angriffs ist es den Inhalt der Datei `secret.log` auszugeben. In dieser Aufgabe wird ein Code-Injection Angriff durch mehrere Umstände erschwert.

- i) Sie können nicht alle System Calls verwenden (wegen seccomp Sandboxing).
- ii) Sie können nur einen sehr kurzen Shellcode direkt einschleusen.

Verwenden Sie einen Shellcode der diese Einschränkungen umgeht. Der Shellcode soll dazu die Datei direkt öffnen und dann auf der Konsole ausgeben. Um die Beschränkung der Länge zu umgehen, verwenden Sie einen Shellcode der aus zwei Teilen besteht. Der erste Teil des Shellcode soll nichts anderes tun als den zweiten Teil in den Speicher zu laden (z.B. mit dem `read` System-Call) und dann zum zweiten Teil des Shellcode zu springen.

Analysieren Sie alle Möglichkeiten um Input in das Programm zu schleusen. Da bei diesem Programm DEP/NX deaktiviert ist, ist jeder User Input potentiell ausführbar und für Shellcode benutzbar. Finden Sie eine Möglichkeit um ihren Shellcode einzuschleusen.

### Aufgabe 4.

- i) Wo befindet sich die Schwachstelle in diesem Programm? Wie können Sie damit die Return-Adresse überschreiben? Wo können Sie Shellcode einschleusen und wie lange kann dieser sein?
- ii) Schreiben Sie einen zweiteiligen Shellcode. Der erste Teil soll den zweiten Teil in den Speicher laden und dann zum zweiten Teil springen (Laden Sie den Shellcode z.B. auf den Stack). Der zweite Teil soll dann die Datei `secret.log` auslesen und auf der Konsole ausgeben.
- iii) Erweitern Sie das Exploit-Template zu einem funktionierenden Angriff. Ihr Angriff soll die erste Zeile oder die ersten 128 Bytes der Datei `secret.log` ausgeben.
- iv) (*Optional*) Finden sie einen Weg mit einem einteiligen Shellcode die Datei `secret.log` auszulesen. Implementieren Sie diesen Angriff in `exploit_alt.py`.